

JavaScript Event Loop

Call Stack, Task Queue, Microtask Queue & Event Loop — Complete Guide

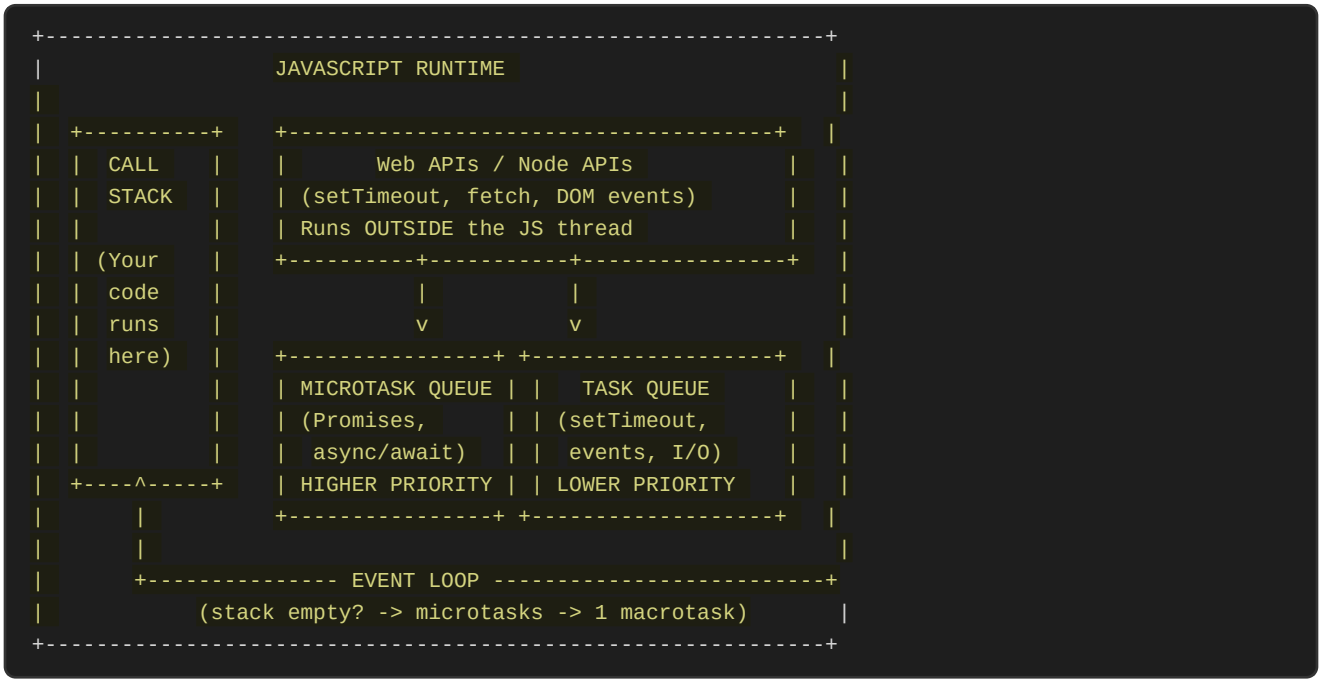
---

How JavaScript handles async code despite being single-threaded

Prepared by Claude for Akshay

1. The Big Picture — How JS Handles Async

JavaScript is **single-threaded** — it can do ONE thing at a time. So how does it handle API calls, timers, and clicks without freezing? Through the **Event Loop system** — a collaboration between 4 key players:



Component	Role	Data Structure
Call Stack	Runs your code (Execution Contexts)	LIFO (Last In First Out)
Web APIs	Handle async work on separate threads	Browser/Node provided
Microtask Queue	Promise callbacks, async/await	FIFO — HIGH priority
Task Queue	setTimeout, events, I/O callbacks	FIFO — LOW priority
Event Loop	Moves callbacks to stack when empty	Continuous loop

2. Call Stack — Where Your Code Runs

The Call Stack is a pile of Execution Contexts. Each function call pushes a new EC on top. When it returns, it's popped off. JS only runs whatever is on **top**.

```
function multiply(a, b) { return a * b; }
function square(n) { return multiply(n, n); }
function printSquare(n) { console.log(square(n)); }
printSquare(5);
// Call Stack trace:
// 1. [Global]
// 2. [printSquare | Global]
// 3. [square | printSquare | Global]
// 4. [multiply | square | printSquare | Global]  <- top
// 5. multiply returns -> popped
// 6. square returns -> popped
// 7. printSquare returns -> popped
```

■ Blocking = Frozen Page

If the stack is busy (heavy computation, infinite loop), NOTHING else can run — no clicks, no animations, no timers. This is why we need async patterns.

3. Web APIs & Task Queue (Macrotasks)

When you call `setTimeout`, `fetch`, or add an event listener, JS hands the job to **Web APIs** that run on separate threads. When done, the callback goes to the **Task Queue**.

```
console.log("1");
setTimeout(() => console.log("2"), 2000);
console.log("3");
// Output: "1", "3", (2 sec later) "2"
// Trace:
// 1. log("1") runs on stack          -> prints 1
// 2. setTimeout -> hands to Web API   -> timer starts
//    setTimeout itself is popped (stack FREE!)
// 3. log("3") runs on stack          -> prints 3
// 4. Stack empty. Timer done.         -> cb to Task Queue
// 5. Event Loop moves cb to stack     -> prints 2
```

What goes into the Task Queue (Macrotasks):

```
Task Queue contains:
  - setTimeout / setInterval callbacks
  - DOM event callbacks (click, scroll, keyup)
  - I/O callbacks (Node.js file/network)
  - MessageChannel
  - setImmediate (Node.js)
```

> ■ **setTimeout(fn, 0) Is NOT Immediate**
>
> It means "run after the current stack empties + all microtasks + your turn in the queue." Browsers enforce a minimum ~4ms delay for nested setTimeout calls.

4. Microtask Queue — The VIP Line

There's a SECOND queue with **higher priority** than the Task Queue. ALL microtasks drain completely before ANY macrotask runs.

```
Microtask Queue contains:
  - Promise .then() / .catch() / .finally()
  - async/await (everything after 'await')
  - queueMicrotask() callbacks
  - MutationObserver callbacks
  - process.nextTick() (Node.js - even higher!)
```

Proof — Promises always beat setTimeout:

```
console.log("1 - Sync");
setTimeout(() => console.log("2 - setTimeout"), 0);
Promise.resolve().then(() => console.log("3 - Promise"));
console.log("4 - Sync");
// Output: 1, 4, 3, 2
//         ^^^^  sync first
//              ^  microtask (Promise) before macrotask
//                 ^  macrotask (setTimeout) last
```

★ **The Critical Rule**

ALL microtasks run before the **NEXT** macrotask. Microtasks can queue more microtasks — and those run too before any macrotask gets a chance. This is why Promises always execute before setTimeout.

5. The Event Loop — Exact Algorithm

```
while (true) {
  // Step 1: Run everything on Call Stack until empty
  executeCallStack();
  // Step 2: Drain ALL microtasks
  while (microtaskQueue.hasItems()) {
    run(microtaskQueue.dequeue());
    // If microtask added MORE microtasks, run those too!
  }
  // Step 3: Render/paint if needed (browser, ~60fps)
  if (timeToRender()) renderUI();
  // Step 4: Take ONE macrotask from Task Queue
  if (taskQueue.hasItems()) {
    run(taskQueue.dequeue());
  }
  // Go back to Step 2! (check microtasks again)
}
```

■ **Priority Order (Highest to Lowest)**

1. **Synchronous code** (Call Stack) — always finishes first
2. **process.nextTick()** (Node.js only) — before other microtasks
3. **Microtask Queue** — Promises, async/await, queueMicrotask — ALL drain
4. **Render/Paint** (browser) — requestAnimationFrame, layout, paint
5. **Task Queue** — setTimeout, events, I/O — ONE per loop cycle

6. Ultimate Test — Complex Example

```
console.log("1");
setTimeout(() => {
  console.log("2");
  Promise.resolve().then(() => console.log("3"));
}, 0);
Promise.resolve().then(() => {
  console.log("4");
  setTimeout(() => console.log("5"), 0);
});
Promise.resolve().then(() => console.log("6"));
setTimeout(() => console.log("7"), 0);
console.log("8");
```

```
// STEP-BY-STEP TRACE:
// --- Sync code runs first ---
// log("1")              -> prints 1
// setTimeout(cbA, 0)     -> cbA to Task Queue
// Promise.then(cbB)      -> cbB to Microtask Queue
// Promise.then(cbC)      -> cbC to Microtask Queue
// setTimeout(cbD, 0)     -> cbD to Task Queue
// log("8")              -> prints 8
//
// State: Output=[1,8]  Micro=[cbB,cbC]  Task=[cbA,cbD]
// --- Drain ALL Microtasks ---
// cbB: log("4")         -> prints 4
```

```
//      setTimeout(cbE)   -> cbE to Task Queue
// cbC: log("6")         -> prints 6
//
// State: Output=[1,8,4,6]  Micro=[]  Task=[cbA,cbD,cbE]
// --- ONE Macrotask ---
// cbA: log("2")         -> prints 2
//      Promise.then(cbF) -> cbF to Microtask Queue
// --- Drain Microtasks ---
// cbF: log("3")         -> prints 3
// --- ONE Macrotask ---  cbD: log("7") -> prints 7
// --- ONE Macrotask ---  cbE: log("5") -> prints 5
// FINAL OUTPUT: 1, 8, 4, 6, 2, 3, 7, 5
```

7. async/await — Where Does It Fit?

`async/await` is sugar over Promises, so it uses the **Microtask Queue**. Code **before** await = synchronous. Code **after** await = microtask.

```
async function example() {
  console.log("A");            // SYNC (runs immediately)
  const result = await Promise.resolve("data");
  console.log("B: " + result);  // MICROTASK (after await)
}
console.log("1");
example();
console.log("2");
// Output: 1, A, 2, B: data
// 'await' pauses the function, rest goes to Microtask Queue
```

What await actually does internally:

```
// This async/await:
async function foo() {
  console.log("A");
  const x = await somePromise;
  console.log("B", x);
}
// Is equivalent to:
function foo() {
  console.log("A");              // sync
  return somePromise.then((x) => {
    console.log("B", x);         // microtask
  });
}
```

Nested async trace:

```
async function foo() {
  console.log("foo start");
  await bar();
  console.log("foo end");     // microtask
}
async function bar() {
  console.log("bar start");
  await Promise.resolve();
  console.log("bar end");     // microtask
}
console.log("script start");
foo();
console.log("script end");
// Output: script start, foo start, bar start,
//         script end, bar end, foo end
```

8. Dangerous Gotchas

■ Gotcha 1: Microtask Starvation

```
function evil() { Promise.resolve().then(evil); }
```

This freezes EVERYTHING — microtasks never stop, so macrotasks and UI rendering never get a chance. Microtasks can queue more microtasks, and they ALL run before moving on.

■ Gotcha 2: Stale Closures in React

```
setCount(count + 1); console.log(count); // still old value!
```

State updates are async (scheduled as microtask-like). The current function still sees the old closure value. Fix: use functional update `setCount(prev => prev + 1)`

■ Gotcha 3: Promise Inside setTimeout vs setTimeout Inside Promise

A Promise created inside a setTimeout runs as a microtask AFTER that macrotask but BEFORE the next macrotask. A setTimeout created inside a Promise runs as a macrotask in a LATER loop cycle. Order matters!

■ Gotcha 4: Node.js process.nextTick()

`process.nextTick()` has HIGHER priority than even Promises. Output order: nextTick > Promise > setTimeout. Abusing nextTick can starve everything else.

9. Starvation & Critical Interview Terms

These are the terms that separate junior from senior developers in interviews. Each one connects back to the Event Loop.

9.1 Starvation

Definition: When one type of task never gets to run because another type keeps hogging the Event Loop.

Type 1: Microtask Starvation (Macrotasks starved)

```
// Microtasks keep queueing more microtasks
// Macrotasks (setTimeout, events) NEVER get to run!
function starve() {
  Promise.resolve().then(() => {
    console.log("microtask");  // runs
    starve();                  // queues another microtask
  });
}
starve();
setTimeout(() => console.log("I NEVER RUN"), 0);
// setTimeout callback is STARVED - microtasks never end
// UI is also frozen - render step never reached!
```

Type 2: I/O Starvation (Node.js)

```
// Heavy computation blocks the event loop
// Incoming HTTP requests pile up, never processed
app.get("/heavy", (req, res) => {
  // This blocks the ENTIRE server for 5 seconds!
  const start = Date.now();
  while (Date.now() - start < 5000) {}
  res.send("done");
});
// During those 5 seconds:
// - ALL other requests wait (starved)
// - No timers fire
// - No I/O callbacks processed
```

Type 3: Render Starvation (UI Jank)

```
// Long-running sync code prevents browser from painting
// User sees frozen/janky UI
button.addEventListener("click", () => {
  // 100,000 DOM updates in one go
  for (let i = 0; i < 100000; i++) {
    document.body.innerHTML += `<div>${i}</div>`;
  }
  // Browser can't render until this finishes!
});
```

✔ **How to Fix Starvation**

1. **Break work into chunks:** Use `setTimeout(nextChunk, 0)` to yield to the event loop
2. **Use requestAnimationFrame:** Sync work with browser's render cycle
3. **Web Workers:** Move heavy computation to a separate thread
4. **Scheduler API:** `scheduler.postTask()` (new) or `requestIdleCallback()`
5. **In Node.js:** Use `setImmediate()` or `worker_threads` for CPU-heavy tasks

```
// FIX: Yielding to the Event Loop (chunking)
function processLargeArray(array) {
  let i = 0;
  function chunk() {
    const end = Math.min(i + 1000, array.length);
    for (; i < end; i++) {
      // process array[i]
    }
    if (i < array.length) {
      setTimeout(chunk, 0);  // yield! let other tasks run
    }
  }
  chunk();
}
```

9.2 Blocking the Event Loop

Definition: Any synchronous code that takes too long, preventing the Event Loop from processing other tasks.

```
// COMMON BLOCKERS:
// 1. Heavy computation
JSON.parse(hugeJSONString);     // blocks until done
// 2. Synchronous I/O (Node.js)
fs.readFileSync("huge.txt");   // blocks entire server!
// 3. Complex regex on large strings
/(a+)+$/.test(longString);      // catastrophic backtracking
// 4. Infinite or very long loops
while (condition) { /* ... */ } // blocks if condition stays true
// 5. alert() / confirm() / prompt() (browser)
alert("This blocks EVERYTHING until dismissed");
```

■ Interview Red Flag

If asked "How can you check if event loop is blocked?" Answer: Use `monitorEventLoopDelay` in Node.js or `PerformanceObserver` with `longtask` in browsers. A healthy event loop iteration should be under 50ms.

9.3 Long Tasks & Jank

Long Task: Any task that takes **>50ms** on the main thread. Causes visible UI jank (stuttering, frozen input, dropped frames).

```
// Detecting Long Tasks in browser
const observer = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    console.log(`Long task: ${entry.duration}ms`);
    // entry.duration > 50ms = jank!
  }
});
observer.observe({ type: "longtask", buffered: true });
```

Duration	User Experience	Classification
0-16ms	Smooth (60fps)	Ideal
16-50ms	Acceptable	Normal
50-100ms	Slight delay felt	Long Task
100-300ms	Noticeable lag	Problematic
300ms+	App feels broken	Critical

9.4 Race Conditions

Definition: When the result depends on the **timing/order** of async operations, leading to unpredictable bugs.

```
// BUG: Race condition in search input
let currentQuery = "";
searchInput.addEventListener("input", async (e) => {
  currentQuery = e.target.value;
  const results = await fetch(`/search?q=${currentQuery}`);
  const data = await results.json();
  // Problem: user typed "React" but slow "Re" response
```

```
  // arrives AFTER fast "React" response
  // UI shows results for "Re" instead of "React"!
  displayResults(data);
});
// FIX: AbortController to cancel stale requests
let controller = null;
searchInput.addEventListener("input", async (e) => {
  if (controller) controller.abort();  // cancel previous!
  controller = new AbortController();
  try {
    const res = await fetch(`/search?q=${e.target.value}`, {
      signal: controller.signal
    });
    displayResults(await res.json());
  } catch (e) {
    if (e.name !== "AbortError") throw e;
  }
});
```

9.5 Zombie Promises

Definition: Promises that **never resolve or reject** — they hang forever, leaking memory and causing silent failures.

```
// BUG: This Promise NEVER settles
const zombie = new Promise((resolve, reject) => {
  // oops - forgot to call resolve() or reject()!
  // some conditional path doesn't resolve
  if (someCondition) {
    resolve("done");
  }
  // What if someCondition is false? ZOMBIE!
});
// Anyone awaiting this hangs forever:
await zombie;  // never continues...
// FIX: Always handle all paths + add timeout
function withTimeout(promise, ms) {
  const timeout = new Promise((_, reject) =>
    setTimeout(() => reject(new Error("Timeout")), ms)
  );
  return Promise.race([promise, timeout]);
}
await withTimeout(fetchData(), 5000);  // fails after 5s
```

9.6 Stack Overflow

Definition: When the Call Stack exceeds its limit (typically ~10,000-15,000 frames) due to deep/infinite recursion.

```
// BUG: No base case = infinite recursion
function count(n) {
  console.log(n);
  count(n + 1);  // never stops!
}
count(1);  // RangeError: Maximum call stack size exceeded
// FIX 1: Add base case
function count(n) {
  if (n > 100) return;  // stop condition
  console.log(n);
  count(n + 1);
}
// FIX 2: Trampoline pattern (avoid stack growth)
function trampoline(fn) {
  let result = fn();
  while (typeof result === "function") result = result();
  return result;
}
function count(n) {
  if (n > 100000) return "done";
  return () => count(n + 1);  // returns fn, not call
}
trampoline(() => count(1));  // no stack overflow!
```

9.7 Callback Hell (Pyramid of Doom)

Definition: Deeply nested callbacks that make code unreadable and error-prone.

```
// THE PYRAMID OF DOOM
```

```
getUser(userId, (user) => {
  getOrders(user.id, (orders) => {
    getOrderDetails(orders[0].id, (details) => {
      getShipping(details.id, (shipping) => {
        console.log(shipping);  // 4 levels deep!
        // Error handling? Good luck.
      });
    });
  });
});
// FIX: async/await (flat, readable, try/catch works)
async function getShippingInfo(userId) {
  const user = await getUser(userId);
  const orders = await getOrders(user.id);
  const details = await getOrderDetails(orders[0].id);
  const shipping = await getShipping(details.id);
  return shipping;
}
```

9.8 Memory Leaks (Event Loop Related)

Definition: Memory that should be freed but isn't — often caused by closures, forgotten timers, and detached DOM nodes.

```
// LEAK 1: Forgotten setInterval (never cleared)
function startPolling() {
  const data = fetchHugeData();  // 100MB
  setInterval(() => {
    console.log(data.status);    // closure keeps data alive!
  }, 1000);
  // interval never cleared -> data never GC'd!
}
// FIX: Always store and clear intervals
const id = setInterval(() => { /* ... */ }, 1000);
clearInterval(id);  // when done
// LEAK 2: Event listeners not removed
function setup() {
  const handler = () => console.log("click");
  button.addEventListener("click", handler);
  // If button is removed from DOM but handler exists
  // -> handler and its closure stay in memory!
}
// FIX: Remove listeners + use AbortController
const ac = new AbortController();
button.addEventListener("click", handler, {
  signal: ac.signal
});
ac.abort();  // removes listener cleanly
// LEAK 3: React useEffect cleanup missing
useEffect(() => {
  const id = setInterval(tick, 1000);
  // MUST return cleanup function!
  return () => clearInterval(id);
}, []);
```

9.9 Throttling & Debouncing

Definition: Rate-limiting techniques that prevent the Task Queue from being flooded with too many event callbacks.

```
// DEBOUNCE: Wait until user STOPS doing something
// Use for: search input, resize, form validation
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
input.addEventListener("input",
  debounce((e) => search(e.target.value), 300)
);
// THROTTLE: Run at most once per interval
// Use for: scroll, mousemove, game loops
function throttle(fn, limit) {
  let waiting = false;
  return (...args) => {
```

```
    if (waiting) return;
    fn(...args);
    waiting = true;
    setTimeout(() => { waiting = false; }, limit);
  };
}
window.addEventListener("scroll",
  throttle(() => updatePosition(), 100)
);
```

Feature	Debounce	Throttle
Fires when	After user STOPS	At regular intervals
Use case	Search input, resize	Scroll, mousemove
Events per 1s (300ms)	1 (after pause)	~3 (every 300ms)
Analogy	Elevator door (waits for last person)	Traffic light (fixed intervals)

9.10 Yielding to the Main Thread

Definition: Intentionally breaking long tasks into smaller chunks so the Event Loop can process other tasks (rendering, user input) between chunks.

```
// METHOD 1: setTimeout(fn, 0) — yield via macrotask
async function heavyWork(items) {
  for (let i = 0; i < items.length; i++) {
    processItem(items[i]);
    if (i % 100 === 0) {
      // Yield every 100 items
      await new Promise(r => setTimeout(r, 0));
    }
  }
}
// METHOD 2: requestAnimationFrame — yield before render
function animateList(items, i = 0) {
  if (i >= items.length) return;
  renderItem(items[i]);
  requestAnimationFrame(() => animateList(items, i + 1));
}
// METHOD 3: scheduler.yield() — modern (Chrome 129+)
async function processData(data) {
  for (const item of data) {
    doWork(item);
    await scheduler.yield();  // cleanest way to yield
  }
}
```

All Interview Terms — Quick Reference

Term	What It Is	How to Fix
Starvation	One task type never runs	Yield, chunk work, Web Workers
Blocking	Sync code hogs the stack	Use async I/O, avoid heavy sync ops
Long Task	Any task > 50ms	Break into chunks, requestIdleCallback
Race Condition	Async timing causes bugs	AbortController, locks, flags
Zombie Promise	Promise never settles	Always resolve/reject + timeout wrapper
Stack Overflow	Too many call stack frames	Base case, iteration, trampoline
Callback Hell	Deeply nested callbacks	async/await, Promise chains
Memory Leak	Memory never freed	Clear timers, remove listeners, cleanup
Debounce	Wait until action stops	Search input, resize events
Throttle	Limit to N times/second	Scroll, mousemove handlers
Yielding	Break long work into chunks	setTimeout(0), rAF, scheduler.yield()

10. Complete Comparison Table

Feature	Call Stack	Microtask Queue	Task Queue
Holds	Execution Contexts	Promise/await callbacks	setTimeout, events, I/O
Order	LIFO	FIFO	FIFO
Priority	Highest (runs first)	2nd (drains completely)	Lowest (one at a time)
Blocking?	YES (busy = frozen)	Can block if infinite	No (one per loop)
Created by	Function calls	.then(), await, queueMicrotask	setTimeout, addEventListener
When runs	Immediately	After stack empties	After stack + ALL microtasks
Per loop cycle	Runs to completion	ALL drain	Only ONE runs

11. Cheat Sheet — Rules to Memorize

```
// The Event Loop Flow:
//
// Your code -> async operation -> Web API handles it
//                                      |
//                                When done, cb goes to:
//                                Promise?  -> MICROTASK Queue
//                                Timer?    -> TASK Queue
//                                      |
//                                EVENT LOOP:
//                                1. Stack empty?
//                                2. Run ALL microtasks
//                                3. Render (maybe)
//                                4. Run ONE macrotask
//                                5. Go to step 2
```

✔ 6 Golden Rules

1. Synchronous code ALWAYS runs first — nothing interrupts the stack
2. ALL microtasks before ANY macrotask — Promises beat setTimeout, always
3. Microtasks can queue more microtasks — they ALL run before next macrotask
4. Only ONE macrotask per loop cycle — then check microtasks again
5. `await` splits a function — before = sync, after = microtask
6. `setTimeout(fn, 0)` means "ASAP after everything else", not instant

■ Quick Priority Order

Sync code > **process.nextTick** (Node) > **Promise/await** > **requestAnimationFrame** (browser) > **setTimeout/events**

★ Interview One-Liners

"JS is single-threaded but non-blocking because of the Event Loop"
"The Event Loop moves callbacks from queues to the stack when it's empty"
"Microtasks have priority over macrotasks — Promises always beat setTimeout"
"await pauses the async function and schedules the rest as a microtask"
"The Call Stack is a LIFO stack of Execution Contexts"